

VERİ YAPILARI VE ALGORİTMALAR

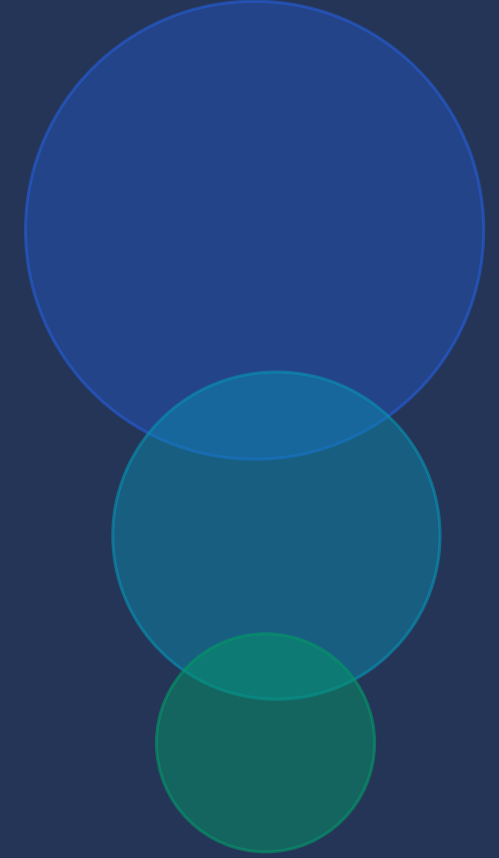
Arama, Sıralama ve Ağaç Yapıları

Hafta 6 · Hafta 7 · Hafta 8 — Kapsamlı Özet

Statik Arama & Hashing

Sıralama Algoritmaları

Ağaç Veri Yapısı



B Ö L Ü M 1

Statik Arama Algoritmaları

Sıralı Arama · İkili Arama · İnterpolasyon · Hash Tabloları (Hafta 6)

Statik Arama – İçindekiler

Bu bölümde ele alınacak konular:

- Sıralı Arama (Linear Search)
- İkili Arama (Binary Search)
- İnterpolasyon Arama (Interpolation Search)
- Hash Tabloları ve Hashing

Statik Veri:

- Arama süresince değişmeyen veri
- Örnek: CD-ROM içerisindeki data

Statik Arama Problemi

- Bilgisayarların önemli kullanım alanlarından biri veriler içerisinde arama yapma işlemidir.

Statik veri:

- Değiştirilemeyen veri — Örn: CD-ROM içerisindeki data

Problemin tanımı:

- Belirtilen bir X verisinin bir A dizisi içerisinde hangi konumda bulunduğunu bulmak
- Bulunmuyorsa bu durumun ifade edilmesi
- Çok sayıda varsa herhangi birinin konumunun belirtilmesi
- A dizisi hiçbir durumda değişmez

Statik Arama Problemi – Örnek

- Telefon rehberindeki bir kişiyi aramak klasik bir statik arama örneğidir.

Verimliliği etkileyen faktör:

- Aranan dizinin sıralanıp sıralanmadığı

İsme göre arama (sıralı veri):

- Telefon defteri isime göre sıralıdır → hızlı arama mümkün

Telefon numarasına göre arama (sırasız veri):

- Numara sıralı değildir → insan için çok zor, lineer arama gerekir

Doğrusal Arama – Sıralı Arama

- Eğer dizi sıralı değilse; eşleşme bulunana kadar diziyi sırayla izleyen doğrusal sıralı arama yapmaktan başka seçenek yoktur.

Algoritmanın karmaşıklığı üç şekilde analiz edilir:

- Başarısız bir aramanın maliyeti
- Başarılı bir aramanın en kötü maliyeti
- Başarılı bir aramanın ortalama maliyeti

Zaman karmaşıklığı: $O(n)$

Doğrusal Arama Algoritması

```
Doğrusal arama( Dizi A, Aranan x)
```

```
Adım 1:  $i = 0$ 
```

```
Adım 2: Eğer  $i > n$  ise Adım 7'ye git
```

```
Adım 3: Eğer  $A[i] = x$  ise Adım 6'ya git
```

```
Adım 4:  $i = i + 1$  yap
```

```
Adım 5: Adım 2'ye git
```

```
Adım 6: Yaz: "Aranan x bulundu. İndis = i" → Adım 8'e git
```

```
Adım 7: Yaz: "Eleman bulunamadı"
```

```
Adım 8: Bitir
```

Doğrusal Arama – Java Kodu

```
// Doğrusal Arama – Java
public static int linearSearch(int[] arr, int x) {

    for (int i = 0; i < arr.length; i++) {

        if (arr[i] == x) {
            return i;    // Bulundu: indisi döndür
        }

    }

    return -1;          // Bulunamadı
}
```

İkili Arama – Binary Search

Temel Özellikler:

- Hızlı bir arama algoritması
- Zaman karmaşıklığı: $O(\log n)$
- Böl – Fethet algoritması
- Ön şart: Veriler sıralı olmalı

Algoritma:

- Veri koleksiyonunun ortadaki elemanı ile belirtilen veriyi karşılaştırır
- Eşleşme gerçekleşirse, orta elemanın indisi döndürülür
- Ortadaki eleman aranandan büyükse, orta elemanın solundaki alt dizide aranır
- Ortadaki eleman aranandan küçükse, orta elemanın sağındaki alt dizide aranır
- Bu işlem alt dizinin boyutu sıfıra düşene kadar devam eder

İkili Arama – Algoritma Adımları

- Adım 1: Listenin ortasından aramaya başla.
- Adım 2: Arananla eşleştirse indisi döndür ve çık.
- Adım 3: Eşleşmediyse değeri incele.
- Adım 4: Aranan orta konumdaki veriden büyükse, daha büyük verileri içeren alt listede arama yap.
- Adım 5: Aranan orta konumdaki veriden küçükse, daha küçük verileri içeren alt listede arama yap.
- Adım 6: $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$ formülü ile yeni orta noktayı bul.
- Adım 7: Eşleşene kadar tekrarla.

İkili Arama – Örnek: Aranan 47

12	21	27	33	43	47	57	59	67	75
0 (low)	1	2	3	4	5	6	7	8	9 (high)

Adım 1: $\text{mid} = 0 + (9 - 0) / 2 = 4 \rightarrow A[4] = 43 \rightarrow 43 < 47 \rightarrow \text{low} = \text{mid} + 1 = 5$

Adım 2: $\text{mid} = 5 + (9 - 5) / 2 = 7 \rightarrow A[7] = 59 \rightarrow 59 > 47 \rightarrow \text{high} = \text{mid} - 1 = 6$

Adım 3: $\text{mid} = 5 + (6 - 5) / 2 = 5 \rightarrow A[5] = 47 \rightarrow \text{BULUNDU!} \checkmark$

→ 3 karşılaştırmayla bulundu. Doğrusal aramada 6 karşılaştırma gerekirdi.

İkili Arama – Java Kodu

```
// İkili Arama – Java
public static int binarySearch(int[] arr, int x) {
    int low = 0;
    int high = arr.length - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == x) return mid;    // Bulundu
        else if (arr[mid] < x) low = mid + 1;
        else high = mid - 1;
    }
    return -1;    // Bulunamadı
}
```

Enterpolasyon Arama

- Enterpolasyon arama, ikili aramanın geliştirilmiş bir çeşididir.

Temel Fikir:

- Gerekli değerin inceleme konumu üzerinde çalışır
- Veriler **sıralı** VE **eşit olarak dağıtılmış** olmalıdır

Avantaj — Telefon Rehberi Örneği:

- 'Mehmet' aranıyorsa → M harfinin olduğu sayfaya doğrudan atlanır
- Doğrusal ve ikili aramadan daha hızlı olabilir

Formül:

- $mid = Lo + ((Hi - Lo) / (A[Hi] - A[Lo])) \times (X - A[Lo])$
- Burada: Lo=en düşük indis, Hi=en yüksek indis, X=aranan değer

Zaman karmaşıklığı: Ortalama $O(\log \log n)$, En kötü $O(n)$

Enterpolasyon Arama – Örnek: Aranan 14

5	8	11	14	17	20	23	26	29	32
0	1	2	3	4	5	6	7	8	9

Formül: $mid = Lo + ((Hi-Lo)/(A[Hi]-A[Lo])) \times (X-A[Lo])$

Verilen: $Lo=0$, $Hi=9$, $A[Lo]=5$, $A[Hi]=32$, $X=14$

$mid = 0 + ((9-0)/(32-5)) \times (14-5)$

$mid = 0 + (9/27) \times 9$

$mid = 0 + 3 = 3$

$A[3] = 14 \rightarrow$ **BULUNDU!**  (Tek adımda!)

Enterpolasyon Arama – Java Kodu

```
// Enterpolasyon Arama – Java
public static int interpolationSearch(int[] arr, int x) {
    int lo = 0;
    int hi = arr.length - 1;

    while (lo <= hi && x >= arr[lo] && x <= arr[hi]) {
        if (arr[lo] == arr[hi]) break;

        int mid = lo + ((hi - lo) / (arr[hi] - arr[lo]))
                    * (x - arr[lo]);

        if (arr[mid] == x) return mid;    // Bulundu
        else if (arr[mid] < x) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;    // Bulunamadı
}
```

Hash Tabloları

- Hash Tablo, ilişkisel bir şekilde veri depolayan bir veri yapısıdır.

Yapı:

- Veriler, her veri değerinin kendine özgü dizin değerine sahip olduğu dizi biçiminde saklanır
- Aranılan verinin indisi bilindiğinde verilere erişim çok hızlıdır

Avantaj:

- Ekleme ve arama işlemleri veri boyutundan bağımsız $\rightarrow O(1)$ idealinde

Depolama:

- Hash Tablo depolama ortamı olarak bir dizi kullanır
- Eleman eklenirken veya aranırken hash tekniği ile indis hesaplanır

Hashing

- Hashing; belirli bir aralıktaki veri değerlerini bir dizinin indis aralığına dönüştürmek için kullanılan bir tekniktir.

Dönüşüm Yöntemi:

- Genellikle mod alma işlemi temelli yöntemler kullanılır
- $\text{hash}(x) = x \% m$ (m: dizinin boyutu)

Kaçı göre mod alınacak?

- Dizinin boyutu kullanılır
- Çakışmaları azaltmak için asal sayı tercih edilir

Başlangıç Durumu:

- Başlangıçta dizinin tüm elemanları NULL olmalı

Değer ile Hashing – Örnek (m=8)

Sıra	Veri	Hash (% 8)	İndis
1	11	$11 \% 8 = 3$	3
2	22	$22 \% 8 = 6$	6
3	13	$13 \% 8 = 5$	5
4	9	$9 \% 8 = 1$	1
5	3	$3 \% 8 = 3$	3 ⚠
6	16	$16 \% 8 = 0$	0
7	19	$19 \% 8 = 3$	3 ⚠
8	1	$1 \% 8 = 1$	1 ⚠

Dizi (indis)

0: 16

1: 9,1

2:

3: 11,3,19

4:

5: 13

6: 22

7:

⚠ Çakışma: farklı değerler aynı indisi hesaplırsa çözüm gerekir.

Değer ile Hashing – Yerleşme Kuralları

- Yerleşme verilerin geliş sırasıyla ilgili DEĞİLDİR.
- Yerleşme verilerin birbirine göre büyük-küçük olma durumuna göre de DEĞİLDİR.
- Yerleşme yalnızca HASHING YÖNTEMİ ile ilgilidir.

Sonuç:

- Aynı formülle hesaplanan her veri, her zaman aynı indise gider
- Bu sayede arama $O(1)$ sürede yapılabilir

Çakışma Durumları:

- $11 \% 8 = 3$, $3 \% 8 = 3$, $19 \% 8 = 3 \rightarrow$ hepsi aynı indise düşer
- $9 \% 8 = 1$, $1 \% 8 = 1 \rightarrow$ aynı indise düşer
- Çakışma çözüm yöntemleri: Zincirleme, Açık Adresleme, Linear Probing(Doğrusal Yoklama)

Anahtar ile Hashing

Verinin kendisi yerine belirtilen bir anahtar ile hashing yapılır. Veri ile birlikte bir anahtar verilir: **(Anahtar, Veri)**
Hash işlemi anahtar değeriyle yapılır → bulunan indise veri yerleştirilir. Veri şifrelemenin temellerindendir.

(Anahtar, Veri)	Hash (% 5)	İndis	Sonuç
(13, Mavi)	$13 \% 5 = 3$	3	Mavi → İndis 3
(9, Bardak)	$9 \% 5 = 4$	4	Bardak → İndis 4
(23, Tokyo)	$23 \% 5 = 3$	3 ⚠	Tokyo → Çakışma!
(16, Sincap)	$16 \% 5 = 1$	1	Sincap → İndis 1
(2475, Akdeniz)	$2475 \% 5 = 0$	0	Akdeniz → İndis 0

Dizi sonucu: [Akdeniz, Sincap, _, Mavi, Bardak]

B Ö L Ü M 2

Sıralama Algoritmaları

Bubble · Selection · Insertion · Merge · Quick Sort (Hafta 7)

Sıralama Problemi

Sıralama nedir?

- Bir listedeki öğeleri artan sırada düzenlemek
- Girdi: n adet sayıdan oluşan dizi $\langle a_1, a_2, \dots, a_n \rangle$
- Çıktı: Girdinin $a'_1 \leq a'_2 \leq \dots \leq a'_n$ şeklinde yeniden düzenlenmesi

Neden Sıralama?

- Arama işlemini kolaylaştırmak
- Birçok algoritmanın altyordamı olarak kullanılır

Sıralama Anahtarı:

- Veri bütünüünün sıralamayı yönlendirmek için seçilmiş özel parçası
- Örn: bir not listesinin isime, numaraya veya nota göre sıralanması

Sıralama Algoritmaları & Özellikler

Örnek Sıralama Algoritmaları:

- Seçimli Sıralama (Selection Sort)
- Kabarcık Sıralama (Bubble Sort)
- Eklemeli Sıralama (Insertion Sort)
- Birleştirmeli Sıralama (Merge Sort)
- Çabuk Sıralama (Quick Sort)

Stabilite (Stability):

- İki eşit elemanın sıralamadan önceki konumlarını koruması
- Sıralamadan önce $i < j$, sıralama sonrasında da $i' < j'$ ise stabil

Yerinde (In-place):

- Fazladan bellek alanı gerektirmeyen sıralama algoritmaları

Temel işlem: Karşılaştırma sayısı

Sıralama Karmaşıklığı – $O(1)$ ve $O(n)$

$O(1)$ — Sabit Zaman:

- Bir dizinin ilk elemanı ile ikinci elemanını birbirleriyle kıyaslayan algoritma
- 10 elemanlı dizi için 1 karşılaştırma
- 10.000 elemanlı dizi için 1 karşılaştırma
- Çalışma zamanı dizinin boyutu ile artmaz

$O(n)$ — Doğrusal Zaman:

- n elemanlı bir dizinin ilk elemanının dizi içindeki herhangi bir elemana eşit olup olmadığını test eden algoritma
- 10 elemanlı için 9 karşılaştırma
- 1000 elemanlı için 999 karşılaştırma
- Çalışma zamanı dizinin boyutu ile doğru orantılı

Sıralama Karmaşıklığı – $O(n^2)$

- n elemanlı bir dizinin herhangi bir elemanının dizi içindeki herhangi başka bir elemana eşit olup olmadığını test eden bir algoritma:

Hesaplama:

- Dizinin 1. elemanının diğer elemanlarıyla karşılaştırılması: $(n-1)$ karşılaştırma
- Dizinin 2. elemanının diğer elemanlarıyla karşılaştırılması: $(n-2)$ karşılaştırma
- Dizinin 3. elemanının diğer elemanlarıyla karşılaştırılması: $(n-3)$ karşılaştırma
- ...
- Toplam: $(n-1)+(n-2)+(n-3)+\dots+2+1 = \frac{n^2}{2} - \frac{n}{2}$ karşılaştırma
- Big Oh $\rightarrow O(n^2)$

Big-Oh Notasyonu – Pratik Etki

- Big-Oh notasyonu, bir algoritmanın çalışma süresinin dizi eleman sayısı ile birlikte nasıl değiştiğine açıklık getirir.

Bir algoritmanın n^2 kıyaslamaya ihtiyacı varsa:

- 4 elemanlı dizi için → 16 kıyaslamaya ihtiyaç vardır
- 8 elemanlı dizi için → 64 kıyaslamaya ihtiyaç vardır
- Dizi eleman sayısı 2 katına çıkarsa, karşılaştırma sayısı 4 katına çıkar!

Örnekler:

- 1.000 eleman → 1.000.000 karşılaştırma
- 10.000 eleman → 100.000.000 karşılaştırma
- 1.000.000 eleman → 1.000.000.000.000 karşılaştırma

Sonuç:

- $O(n^2)$ algoritmalar büyük veri setleri için kabul edilemez derecede yavaştır

Kaba Kuvvet (Brute Force) Sıralama

Kaba Kuvvet Sıralama Algoritmaları:

- Kabarcık (Bubble) Sıralama
- Seçimli (Selection) Sıralama

Ortak Özellikler:

- Her ikisi de basit fakat verimsizdir
- Her ikisi de $O(n^2)$ zaman karmaşıklığına sahiptir
- Yerinde (in-place) sıralama yaparlar
- Küçük veri setleri için kabul edilebilir
- Büyük veri setleri için tercih edilmez

Kabarcık (Bubble) Sıralama

- Basit fakat verimsiz bir sıralama algoritması.

Algoritma:

- 1. Dizinin tüm elemanlarını sırayla ele alır
- 2. Ele aldığı elemanla kendisinden sonraki elemanı karşılaştırır
- 3. Ele aldığı eleman bir sonraki elemandan büyükse yerlerini değiştirir, küçükse olduğu gibi bırakır
- 4. Bir sonraki elemana geçer
- Her geçişte en büyük eleman sona 'kabarcık gibi' çıkar

Karmaşıklık:

- En iyi: $O(n)$ | Ortalama: $O(n^2)$ | En kötü: $O(n^2)$

Stabil:  | Yerinde: 

Kabarcık Sıralama – İterasyon Adımları

Başlangıç:

89	45	68	90	29
----	----	----	----	----

45	89	68	90	29
----	----	----	----	----

i=2 sonrası:

45	68	89	90	29
----	----	----	----	----

i=3 sonrası:

45	68	89	29	90
----	----	----	----	----

45	68	29	89	90
----	----	----	----	----

i=4 sonrası:

45	29	68	89	90
----	----	----	----	----

i=5 — Sıralı ✓:

29	45	68	89	90
----	----	----	----	----

Seçimli (Selection) Sıralama

- Basit fakat verimsiz bir sıralama algoritması.

Algoritma:

- 1. İlk iterasyonda dizideki en küçük elemanı bulur ve ilk elemanla yer değiştirir
- 2. İkinci iterasyonda ikinci elemandan itibaren en küçük elemanı bulur ve ikinci elemanla yer değiştirir
- 3. Sondan bir önceki elemanla son elemanın kıyaslanmasına kadar devam eder
- 4. i. iterasyondan sonra, dizinin en küçük i elemanı ilk i sıraya yerleşir

Karmaşıklık:

- En iyi: $O(n^2)$ | Ortalama: $O(n^2)$ | En kötü: $O(n^2)$

Stabil: ❌ | Yerinde: ✅

Seçimli Sıralama – İterasyon Adımları

Başlangıç:

89	45	68	90	29
----	----	----	----	----

i=0: 29↔89:

29	45	68	90	89
----	----	----	----	----

i=1: 45 zaten:

29	45	68	90	89
----	----	----	----	----

i=2: 68 zaten:

29	45	68	90	89
----	----	----	----	----

i=3: 89↔90 ✓:

29	45	68	89	90
----	----	----	----	----

Eklemeli (Insertion) Sıralama

- Azalt-Fethet algoritması — basit fakat verimsiz.

Algoritma:

- 1. İlk iterasyonda dizinin 2. elemanını alır, 1. elemandan küçükse yer değiştirir
- 2. 2. iterasyonda 3. elemana bakar, önceki iki elemana göre uygun yere yerleştirir; böylece üç eleman da sıralanmış olur
- 3. Algoritmanın i . iterasyonunda, ilk i eleman doğru şekilde sıralanmış olacaktır

Durum Analizi:

- En kötü: Dizi ters sıralı $\rightarrow O(n^2)$
- En iyi: Dizi zaten sıralı $\rightarrow O(n)$
- Ortalama: Dağınık dizi $\rightarrow O(n^2)$

Stabil:  | Yerinde: 

Eklemeli Sıralama – Karmaşıklık Analizi

- Eklemeli sıralama algoritması, $O(n^2)$ tipinde bir algoritmadır.

For döngüsü:

- $(n-1)$ kere çalışır
- Elemanı sıralanmış elemanlar içindeki doğru yerine yerleştirir

While döngüsü:

- Dizinin kalan elemanları üzerinde işlemini gerçekleştirir

İterasyon Sayıları:

- İlk aşamada: $(n-1)$ iterasyon gerçekleştirilir
- İkinci aşamada: $(n-2)$ iterasyon gerçekleştirilir
- Toplam $\rightarrow$ Big Oh = $O(n^2)$

Durum Analizi:

- En kötü: Ters sıralı dizi
- En iyi: Sıralı dizi
- Ortalama: Dağınık dizi

Eklemeli Sıralama – [89, 45, 68, 90, 29, 34]

Başlangıç:

89	45	68	90	29	34
----	----	----	----	----	----

i=1: 45↔89:

45	89	68	90	29	34
----	----	----	----	----	----

i=2: 68 yeri:

45	68	89	90	29	34
----	----	----	----	----	----

i=3: 90 yeri:

45	68	89	90	29	34
----	----	----	----	----	----

i=4: 29 yeri:

29	45	68	89	90	34
----	----	----	----	----	----

i=5 ✓:

29	34	45	68	89	90
----	----	----	----	----	----

Birleştirmeli (Merge) Sıralama

- Böl-Fethet Algoritması — verimli fakat daha karmaşık.

Algoritma:

- 1. Diziyi eşit uzunluklu iki diziye böler
- 2. Eleman sayısı tek sayıysa, birinin eleman sayısı diğerinden bir fazla olur
- 3. Her diziyi kendi içinde yinelemeli (recursive) olarak sıralar
- 4. Sıralı iki diziyi tek bir dizide birleştirir

Rekürsif Tanım:

- Temel durum: tek elemanlı dizi — zaten sıralı, hemen geri döner
- Yineleme adımı: iki veya daha fazla elemanlı diziyi iki eşit alt diziye böler

Karmaşıklık:

- En iyi / Ortalama / En kötü: $O(n \log n)$ | Alan: $O(n)$

Stabil:  | Yerinde: 

Merge Sort – Bölme ve Birleştirme Süreci

8 3 2 9 7 1 5 4

8 3 2 9

7 1 5 4

8 3

2 9

7 1

5 4

3 8

2 9

1 7

4 5

2 3 8 9

1 4 5 7

1 2 3 4 5 7 8 9 ✓

Merge Sort – Adım Adım Açıklama

- Örnek dizi: [8, 3, 2, 9, 7, 1, 5, 4]

Bölme (Divide):

- [8,3,2,9] ve [7,1,5,4]
- [8,3] [2,9] ve [7,1] [5,4]
- [8] [3] [2] [9] ve [7] [1] [5] [4] — tek elemanlı, durma

Birleştirme (Merge — sıralayarak):

- [3,8] [2,9] ve [1,7] [4,5]
- [2,3,8,9] ve [1,4,5,7]
- [1,2,3,4,5,7,8,9] ✓

Toplam karşılaştırma: $O(n \log n) = O(8 \times 3) \approx 24$

Çabuk (Quick) Sıralama

- Böl ve Fethet sıralama algoritması.

Merge Sort'tan Farkı:

- Diziyi alt dizilere konumlarına göre değil, değerlerine göre böler

Algoritma:

1. Bir eleman pivot seçilir
2. Dizi; pivot, pivottan küçükler ve pivottan büyükler olmak üzere 3 alt kısma ayrılır
3. Diğer iki alt dizi arasında pivottan küçük elemanlar ile pivottan büyük elemanlar değiştirilir
4. Pivotun solunda pivottan büyük elemanlar olacak şekilde yer değişimi yapılır
5. Her alt dizi için yinelemeli olarak aynı işlem uygulanır

Çabuk Sıralama – Karmaşıklık Analizi

En iyi durum: $O(n \log n)$

- Pivot her seferinde diziyi iki eşit parçaya böler

Ortalama durum: $O(n \log n)$

- Rastgele pivot seçimi ile elde edilir

En kötü durum: $O(n^2)$

- Pivot her seferinde en küçük veya en büyük eleman seçilirse
- Örneğin zaten sıralı bir dizide ilk eleman pivot seçilirse

Bellek:

- $O(\log n)$ yığın (stack) alanı — rekürsif çağrılar için

Stabil:  | Yerinde: 

Pratikte çok hızlıdır — en popüler sıralama algoritmalarından biridir

Çabuk Sıralama – Pivot İşlemi Örneği

Başlangıç
Pivot=45

89	45	68	90	29
	↑pivot			

Küçükler | Pivot | Büyükler

29	45	89	68	90
	↑pivot			

Sol: [29] sıralı

29	45	89	68	90
----	----	----	----	----

Sağ: pivot=89

29	45	68	89	90
			↑pivot	

Sonuç ✓

29	45	68	89	90
----	----	----	----	----

Sıralama Algoritmalarının Karşılaştırılması

Algoritma	En İyi	Ortalama	En Kötü
Kabarcık	$O(n)$	$O(n^2)$	$O(n^2)$
Seçmeli	$O(n^2)$	$O(n^2)$	$O(n^2)$
Eklemeli	$O(n)$	$O(n^2)$	$O(n^2)$
Birleştirmeli	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Çabuk	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

B Ö L Ü M 3

Ağaç Veri Yapısı

Temel Kavramlar · İkili Ağaç · Gezinme · Uygulamalar (Hafta 8)

Ağaç Veri Yapısı – Giriş

- Ağaç; bir kök işaretçisi, sonlu sayıda düğümleri ve onları birbirine bağlayan dalları olan bir veri modelidir.

Hiyerarşik Yapı:

- Aile soyağacındaki gibi hiyerarşik bir yapıya sahiptir
- Çocuk, kardeş düğüm, aile, ata gibi kavramlar ağaç modelinde de kullanılır

Farklı Ağaç Türleri:

- İkili ağaç (Binary Tree)
- Kodlama ağacı (Huffman Tree)
- Sözlük ağacı
- Kümeleme ağacı
- Uygulamaya yönelik özel ağaç şekilleri de türetilebilir

Ağaç – Doğrusal Olmayan Yapı

- Bağlı listeler, yığınlar ve kuyruklar doğrusal (linear) veri yapılarıdır.
- Ağaçlar ise doğrusal olmayan, belirli niteliklere sahip iki boyutlu veri yapılarıdır.

Bileşenler:

- Her ağaç düğümlerden (node) ve kenarlardan (edge) oluşur
- Her düğüm bir nesneyi gösterir
- Her kenar (bağlantı) iki düğüm arasındaki ilişkiyi gösterir

Avantaj:

- Arama işlemi bağlı listelere göre çok daha hızlı yapılır

Ağaç – Düğüm Bağlantı Yapısı

- Ağacın düğümlerindeki bilgiler sayılardan (veya herhangi bir veri türünden) oluşmuştur.

Düğüm Bağlantıları:

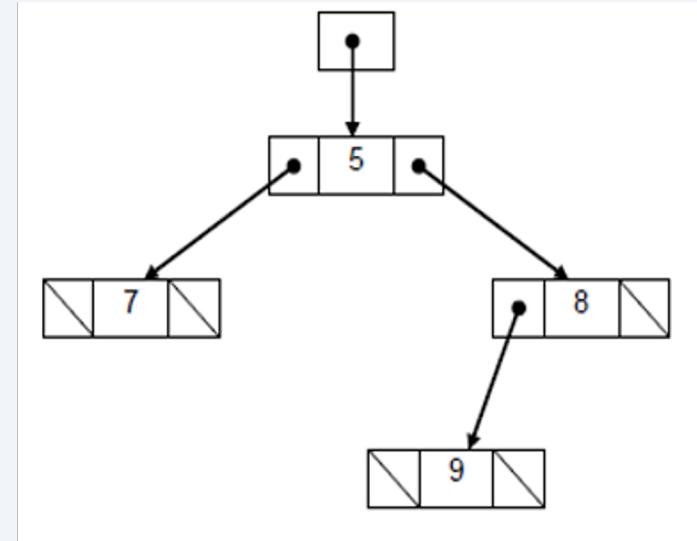
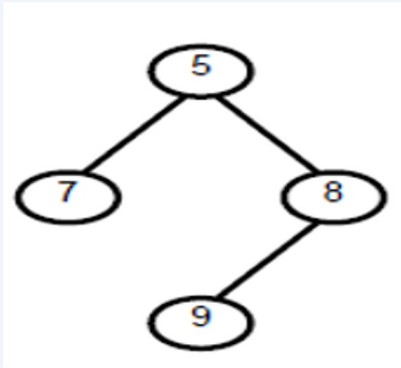
- Her düğümdeki sol ve sağ bağlar yardımıyla diğer düğümlere ulaşılır
- Sol veya sağ bağ null olabilir mi? → Evet

Örnek Yapı:

- Kök düğüm → sol alt ağaç ve sağ alt ağaç
- Her yaprak düğümün sol ve sağ bağı null'dır

Not:

- Ağacın düğüm sayısı = n ise kenar sayısı = $n-1$ 'dir

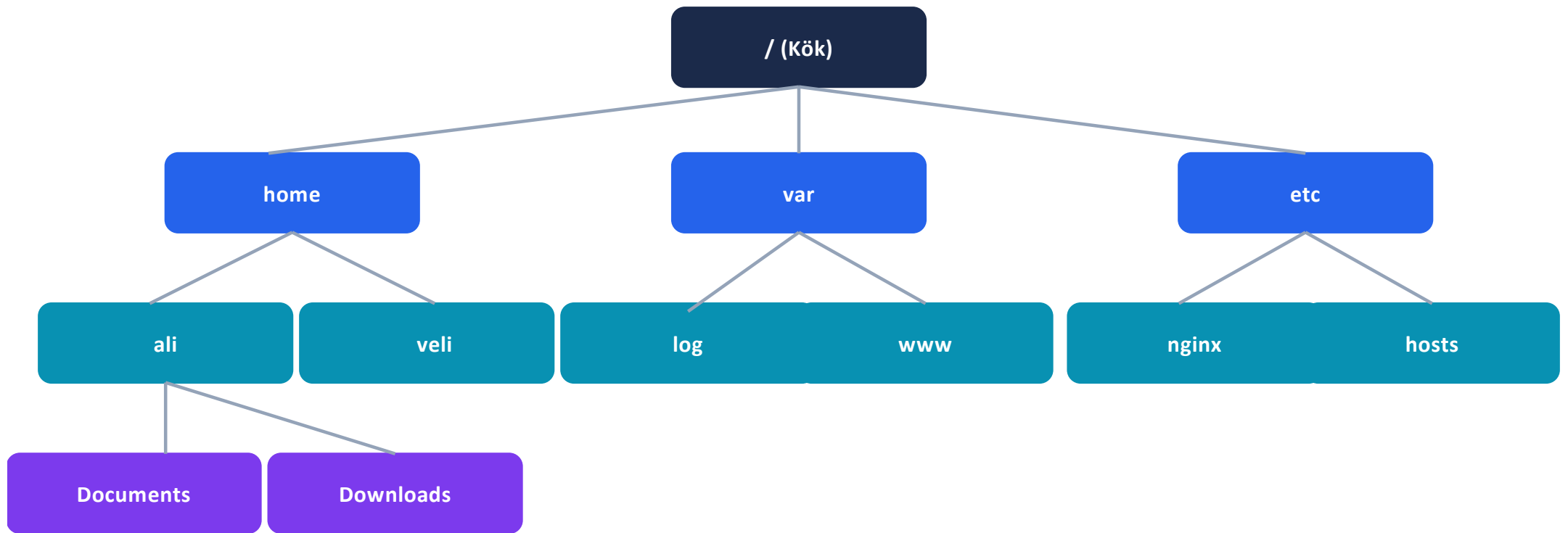


Ağaç – Yazılım Uygulama Alanları

Ağaçların bilgisayar bilimlerinde uygulama alanları:

- İşletim sisteminin dosya sistemini modellemek
- Oyunlar için farklı hamleleri ele almak
- Ağ routing (yönlendirme) algoritmalarında
- Dinamik yazım kontrolü gibi alanlarda
- Huffman sıkıştırma kodlamasında
- Derleyicilerde matematiksel ifadeleri modellemede

Örnek Ağaç Yapısı – Dosya Sistemi



Ağaç – Temel Kavramlar (1)

- A düğümü kök olmak üzere 7 düğümden (n) oluşan ağaçta toplam 6 kenar (n-1) vardır.

Düğüm (Node):

- Ağacın her elemanına verilen isim

Kök (Root):

- Ağacın başlangıç düğümü; üzerinde başka düğüm yoktur

Çocuk (Child):

- Bir düğüme doğrudan bağlı olan düğümler

Kardeş Düğüm (Sibling):

- Aynı ebeveyn düğüme bağlı düğümler

Aile (Parent):

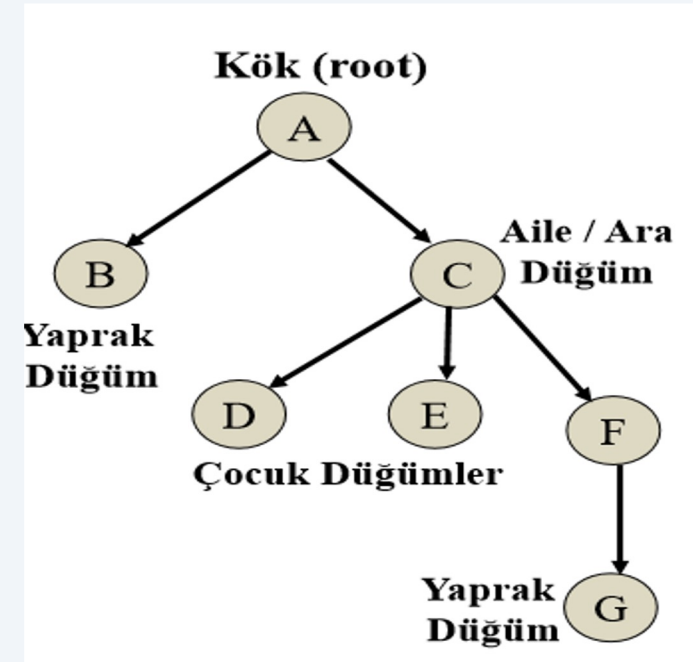
- Düğümlerin doğrudan bağlı oldukları üst düğüm

Ata (Ancestor):

- Ebeveyn düğümünün daha üstünde kalan düğümler

Torun (Descendant):

- Bir düğümün çocuğuna bağlı olan tüm düğümler



Ağaç – Temel Kavramlar (2)

Yaprak (Leaf):

- Ağacın en altında bulunan, çocukları olmayan düğümler

Derece (Degree):

- Bir düğümden alt hiyerarşiye yapılan bağlantıların sayısı (çocuk sayısı)

Seviye (Level):

- Hiyerarşik sıra (rank). Kök düğüm seviye = 0

Derinlik (Depth):

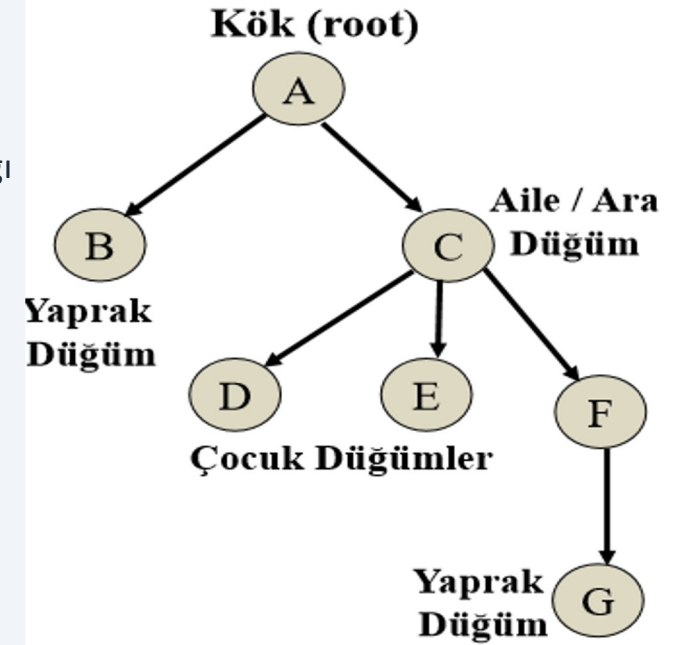
- Bir düğümün köke olan uzaklığı. Kök düğüm derinlik = 0

Yükseklik (Height):

- Bir düğümün kendi silsilesinden en uzak yaprak düğüme olan uzaklığı
- Yaprak düğümlerin yüksekliği = 0 | Kök yüksekliği = Ağaç Yüksekliği

Yol (Path):

- Bir düğümden başka bir düğüme ulaşmak için üzerinden geçilen düğümlerin listesi

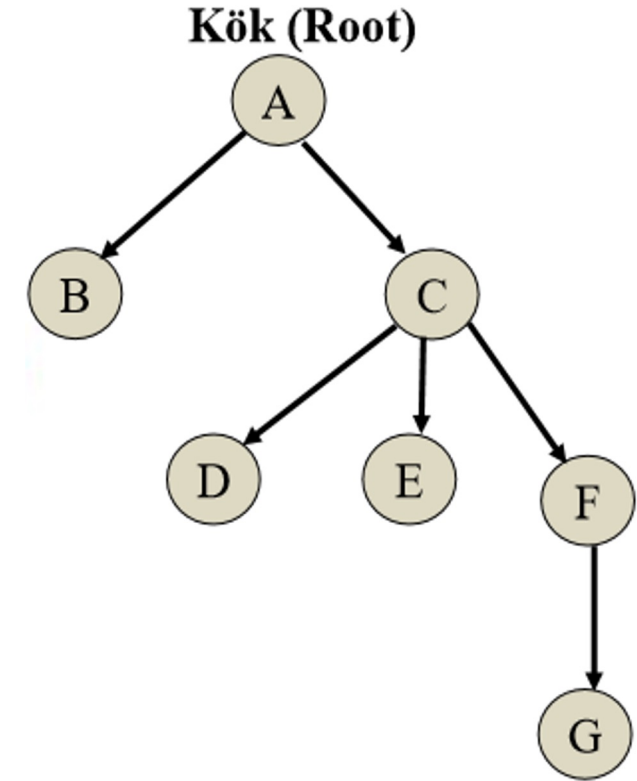


Temel Kavramlar – Tablo Örneği

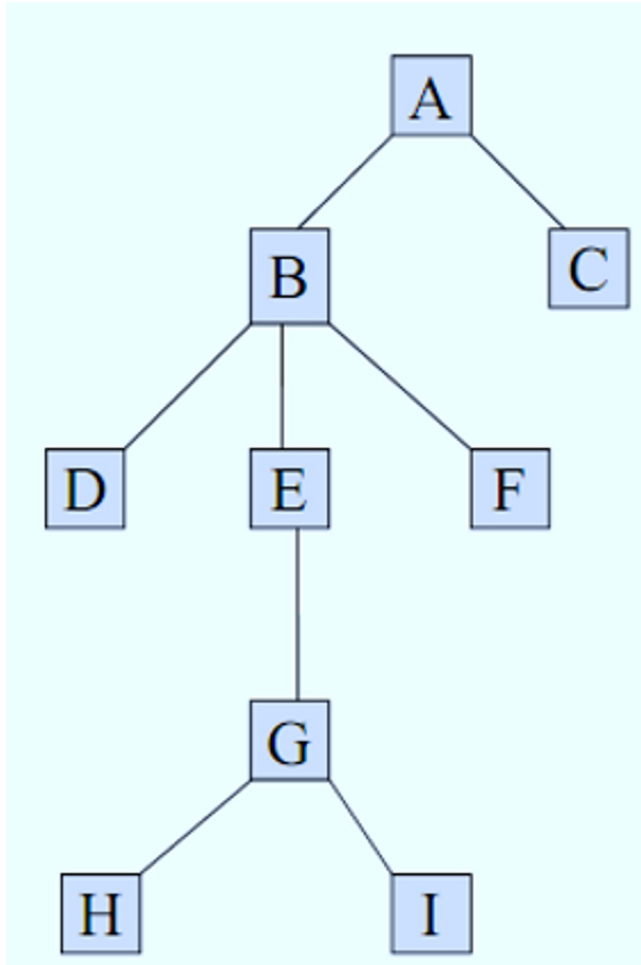
Sol alt ağaç B kökü, sağ alt ağaç C kökü ile başlar. A'dan solda B'ye, sağda C'ye iki dal çıkar.

Tanım	Kök (A)	D düğümü
Çocuk / Derece	2	0
Kardeş	1	3
Düzey	0	2
Aile (Parent)	Yok	C
Ata (Ancestor)	Yok	Kök
Yol	A	$A \rightarrow C \rightarrow D$
Derinlik	0	2
Yükseklik	3	0 (yaprak)

Düğüm sayısı:9 | Yükseklik:4 | Kök:A | Yapraklar:C,D,F,H,I | H'nin ataları:E,B,A | B'nin torunları:G,H,I | E'nin kardeşleri:D,F



Temel Kavramlar – Tablo Örneği



- Düğüm sayısı:
 - 9
- Yükseklik:
 - 4
- Kök düğüm:
 - A
- Yapraklar:
 - C,D,F,H,I
- Seviye sayısı:
 - 4
- H'nin ataları:
 - E,B,A---?????
- B'nin torunları:
 - G,H,I
- E'nin kardeşleri:
 - D,F

Rekürsif Ağaç Tanımı

- Rekürsif mantıkla bir ağaç, bir kök (root) ve alt-ağaçlar şeklinde tanımlanabilir.

Tanım:

- Boş ağaç, veya
- Bir kök düğüm + kökün çocuklarının alt ağaçlarından oluşan yapı

Örnek:

- 1 düğümü: 2 ve 3 alt-ağaçlarından oluşur
- 2 düğümü: 4, 6 ve 5(5-9-10) olmak üzere üç alt-ağaçtan oluşur

Önemi:

- Tüm ağaç işlemleri (gezinme, arama, silme, ekleme) rekürsif tanımlanır
- Temel durum: tek düğümlü ağaç veya boş ağaç

İkili Ağaç (Binary Tree)

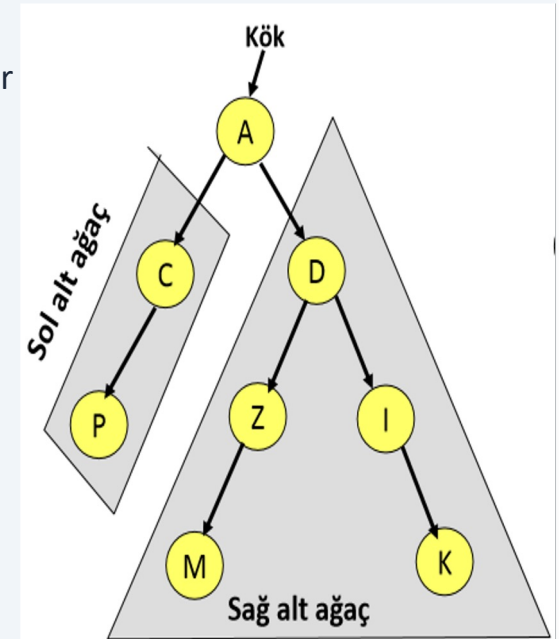
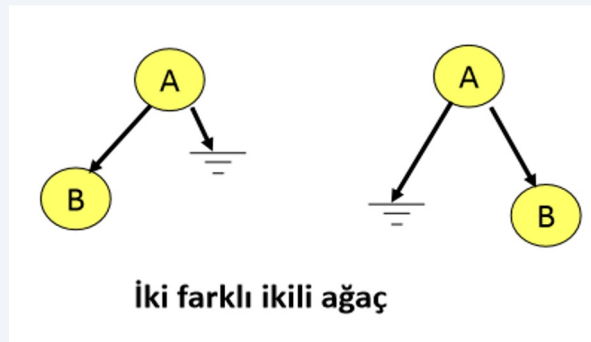
- Her düğümün en fazla iki çocuk düğüme sahip olduğu ağaç yapısıdır.
- Bilgisayar bilimlerinde en çok kullanılan ağaç veri yapısıdır.
- Sıralı olması durumunda arama, ekleme ve silme işlemlerini oldukça hızlandırır.

İkili Ağaç yapısı:

- Boş olabilir
- Tek bir düğümden oluşabilir
- Sol alt ağaç ve sağ alt ağaç olmak üzere köke ait iki adet ikili ağaçtan oluşabilir

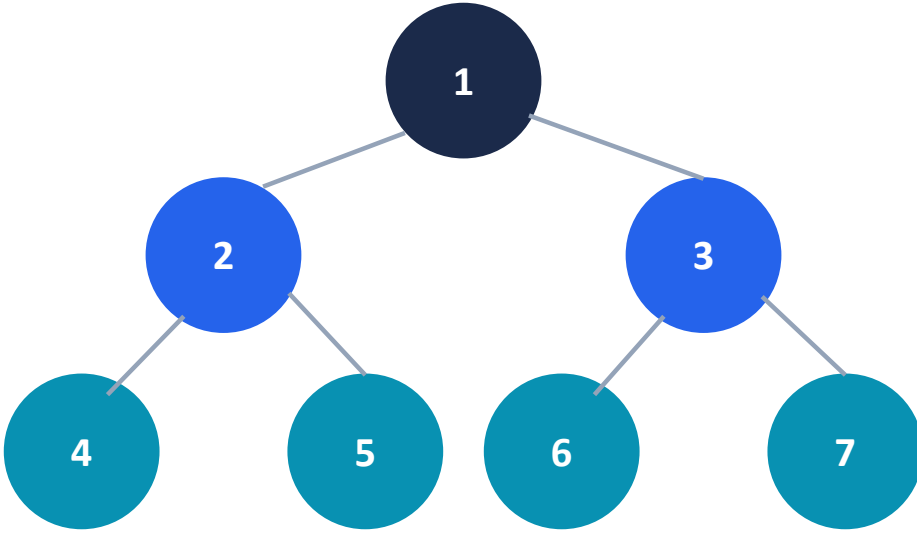
En önemli şart:

- İlgili ikili ağaç düğümünün en fazla iki çocuğa (alt ağaca) sahip olabilmesi



İkili Ağaç – Görsel Yapı

İkili Ağaç Örneği:



Özellikler

- Kök: 1 (en üstteki)
- Seviye 0: {1}
- Seviye 1: {2, 3}
- Seviye 2: {4, 5, 6, 7}
- Yapraklar: 4, 5, 6, 7
- Yükseklik: 2
- Düğüm sayısı: 7
- Kenar sayısı: 6 ($n-1$)
- Bu tam ikili ağaçtır ✓

Katı İkili Ağaç (Strict / Proper Binary Tree)

- Yaprak düğümler haricindeki tüm düğümler sıfır veya iki çocuğa sahipse 'katı ikili ağaç' adını alır.

Kural:

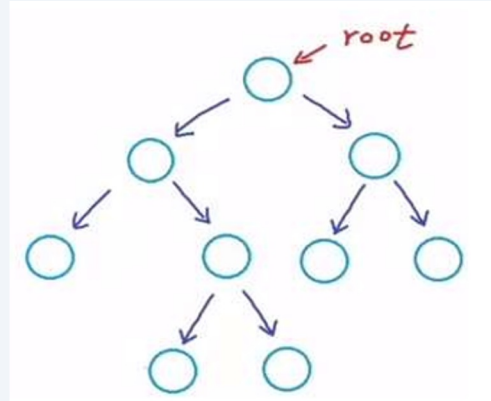
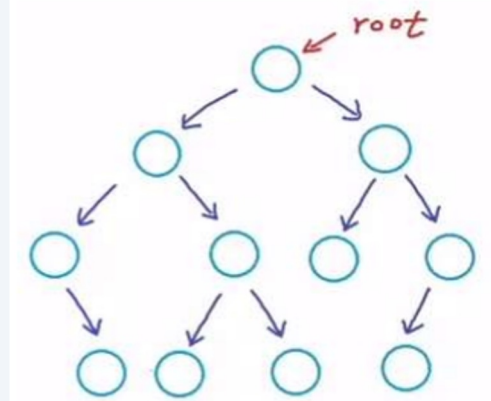
- Her iç düğüm ya 0 ya da 2 çocuğa sahiptir
- Tek çocuklu iç düğüm bulunmaz

Katı İkili Ağaç mı? Kontrol:

- Tüm iç düğümlerin çocuk sayısı 0 veya 2 ise → EVET
- Herhangi bir iç düğümün 1 çocuğu varsa → HAYIR

Özellik:

- n iç düğüm varsa, yaprak sayısı = $n + 1$
- Toplam düğüm sayısı = $2n + 1$ (n iç + $n+1$ yaprak)



Tam İkili Ağaç (Full Binary Tree)

- Her bir düğümün net olarak iki çocuk düğüme sahip olduğu ve yaprak düğümlerin aynı seviyede olduğu ikili ağaçtır.

Kurallar:

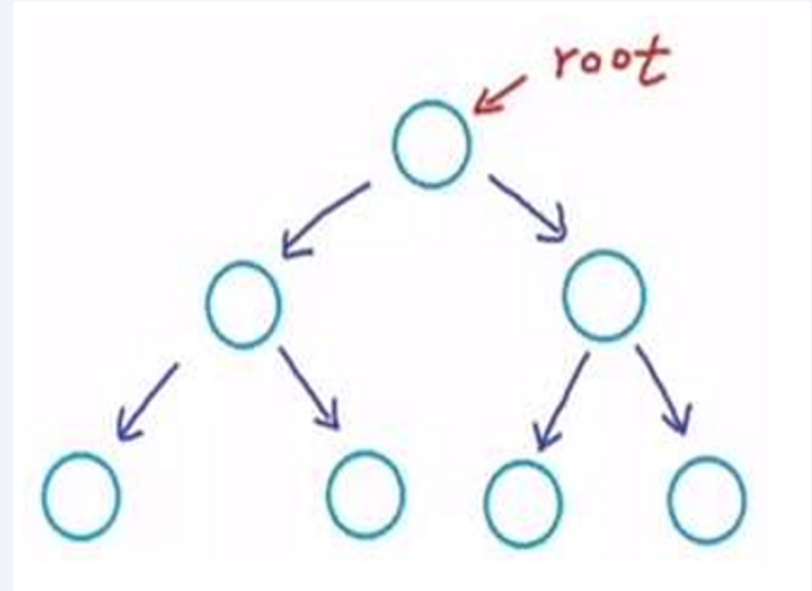
- Her düğüm tam olarak 2 çocuğa sahiptir
- Her yaprak aynı derinlikte (seviyede) olmalı
- Her düğüm eşit şekilde sağ ve sol alt-ağaçlara sahiptir

Matematiksel Özellikler:

- h yüksekliğindeki tam ağaçta yaprak sayısı $= 2^h$
- h yüksekliğinde toplam düğüm sayısı $= 2^{(h+1)} - 1$

Sonuç:

- Mükemmel dengeli yapı
- Hem katı hem de eksiksiz ikili ağaçtır



Eksiksiz İkili Ağaç (Complete Binary Tree)

- Son seviye dışındaki tüm seviyelerin tam (full) olduğu ikili ağaç türüdür.

Kurallar:

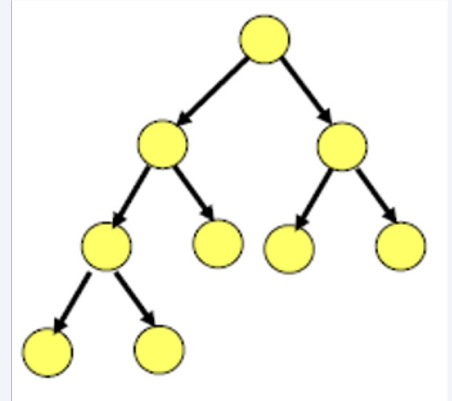
- Düğümleri sol taraftan doldurulur
- Yeni bir derinliğe soldan sağa doğru ekleme başlanır
- Son seviye soldan sağa dolu olmalıdır (sağda boşluk olabilir)

Tam ağaçtan farkı:

- Tam ağaçta tüm yapraklar aynı derinliktedir
- Eksiksiz ağaçta son katman eksik olabilir ama sola yaslanmış olmalı

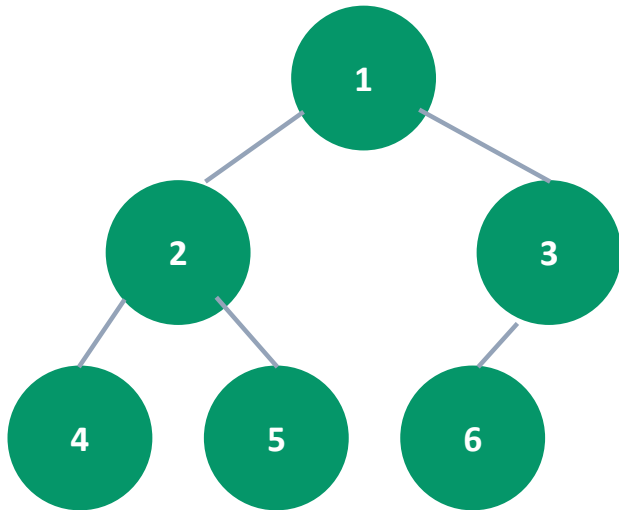
Kullanım:

- Heap (yığın) veri yapısı eksiksiz ikili ağaç kullanır
- Dizi ile verimli şekilde saklanabilir: i . düğümün çocukları $2i$ ve $2i+1$. indislerdedir



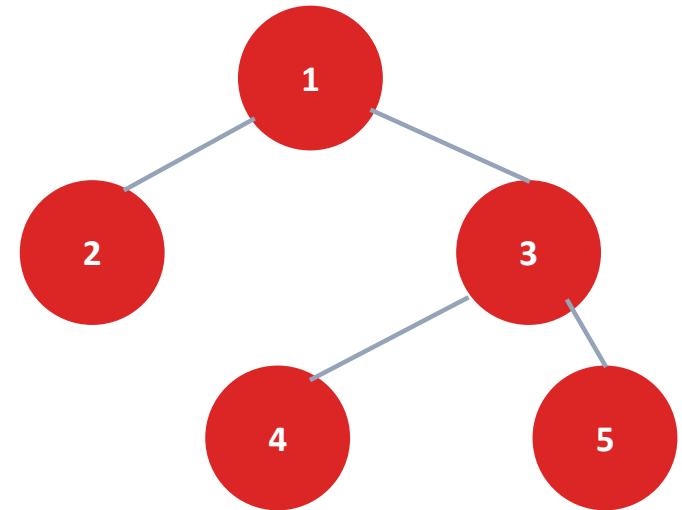
Eksiksiz İkili Ağaç – Örnek Kontrol

Eksiksiz İkili Ağaç ✓



Son seviyede 6 solda ✓
Sağda boşluk kabul edilir

Eksiksiz DEĞİL ✗



4 sağda, 5 solda değil!
Soldan doldurmak zorunlu

İkili Ağaç Özellikleri – 1

Özellik 1: Seviye k 'da maksimum düğüm sayısı = 2^k

- $k=0$ (kök): maksimum $2^0 = 1$ düğüm
- $k=1$: maksimum $2^1 = 2$ düğüm
- $k=2$: maksimum $2^2 = 4$ düğüm
- $k=3$: maksimum $2^3 = 8$ düğüm

Özellik 2: h yüksekliğinde maksimum düğüm sayısı = $2^{h+1} - 1$

- $h=0$: $2^1 - 1 = 1$ düğüm
- $h=1$: $2^2 - 1 = 3$ düğüm
- $h=2$: $2^3 - 1 = 7$ düğüm
- $h=3$: $2^4 - 1 = 15$ düğüm

Örnek: $h=3 \rightarrow$ maksimum düğüm sayısı = $16 - 1 = 15$

İkili Ağaç Özellikleri – 2

Özellik 3: h yüksekliğinde minimum düğüm sayısı = $h + 1$

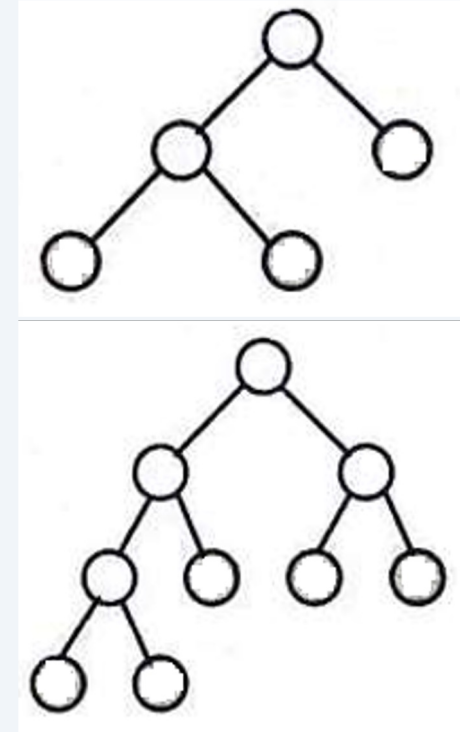
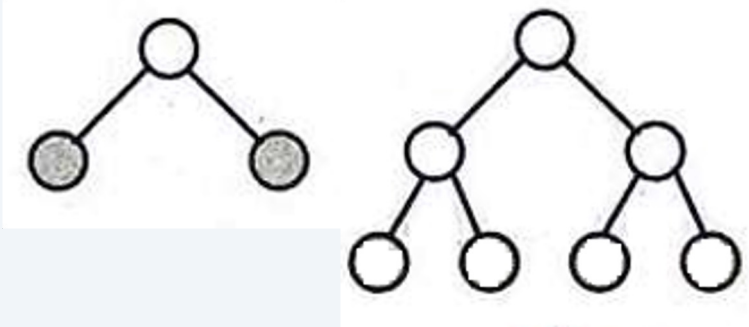
- Kök düğümün $h=0$ 'da olduğuna dikkat
- $h=0$: minimum 1 düğüm (sadece kök)
- $h=3$: minimum 4 düğüm (her seviyede 1 düğüm)

Özellik 4: n düğümlü ağaçta kenar sayısı = $n - 1$

- En az 2 düğüme sahip olması gerekir
- 4 düğümlü örneklerde 3 kenar bulunur

Özellik 5: n yapraklı eksiksiz ağaçta kenar sayısı = $2(n-1)$

- $n=2 \rightarrow k=2$
- $n=3 \rightarrow k=4$
- $n=4 \rightarrow k=6$
- $n=5 \rightarrow k=8$



İkili Ağaç Oluşturma Yöntemleri

Dizi Kullanarak:

- Eksiksiz ikili ağaç için verimli
- i . düğümün sol çocuğu $\rightarrow$ indis $2i$
- i . düğümün sağ çocuğu $\rightarrow$ indis $2i+1$
- i . düğümün ebeveyni $\rightarrow$ indis $\lfloor i/2 \rfloor$
- Basit ama esnek değil; silme-ekleme zor

Bağlı Liste Kullanarak:

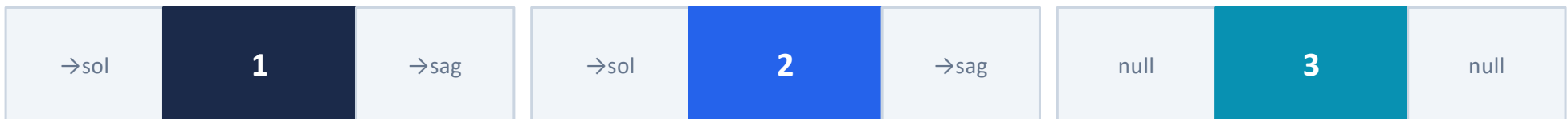
- Her düğümde: veri, sol işaretçi, sağ işaretçi
- Dinamik bellek $\rightarrow$ boyut önceden bilinmesi gerekmez
- Ekleme ve silme işlemleri daha esnektir

Bağlı Liste İkili Ağaç Gerçekleştirimi (1)

```
// Java'da düğüm – iç sınıf olarak tanımlanır
class Node {
    int data;           // Düğümün verisi
    Node left, right;   // Sol ve sağ çocuk (null olabilir)

    Node(int data) { this.data = data; } // Yapıcı
}
```

Bellek Temsili – Her düğüm 3 alan içerir:



Kök → [null | 1 | →] ←sol [null|2|→] sağ→ [null|3|null]

Sol veya sağ referans **null** olabilir → o yönde çocuk yok demektir. Java'da referanslar varsayılan olarak null'dır.

Bağlı Liste İkili Ağaç Gerçekleştirimi (2)

```
// Java'da İkili Ağaç sınıfı
class BinaryTree {
    Node root; // Kök referansı (başlangıçta null)

    // Yeni düğüm oluşturma
    Node createNode(int value) {
        return new Node(value); // Java GC yönetir
    }
}
```

```
// main içinde ağaç oluşturma
BinaryTree tree = new BinaryTree();
tree.root = tree.createNode(1); // Kök
tree.root.left = tree.createNode(2); // Sol çocuk
tree.root.right = tree.createNode(3); // Sağ çocuk
tree.root.left.left = tree.createNode(4);
tree.root.left.right = tree.createNode(5);
```

Bağlı Liste İkili Ağaç Gerçekleştirimi (3)

Dizi vs Bağlı Liste Karşılaştırması:

- Dizi: indis hesabı ile hızlı erişim, ama sabit boyut
- Bağlı liste: dinamik boyut, referans yönetimi gerekli

Java'da Temel İşlemler:

- Ekleme (Insertion): Uygun konumu bul, new Node(value) oluştur, referansları ayarla
- Silme (Deletion): Düğümü bul, çocukları yeniden bağla — belleği GC temizler
- Arama (Search): Rekürsif ya da iteratif kökten başlayarak ilerle

Java'da Bellek Yönetimi:

- C++'de: new/delete manuel yönetim gerekir
- Java'da: Garbage Collector (GC) referanssız nesneleri otomatik temizler
- delete çağrısı yoktur — referansı null yapman yeterli

Ağaç yüksekliği ne kadar büyürse rekürsif çağrılar o kadar derin yığın (stack) kullanır

İkili Ağaç Üzerinde Gezinme

? Kökten itibaren düğümleri elimizde olan ikili ağaç üzerinde nasıl dolaşırsınız?

- Bir ağacı bir düğüme sadece bir defa uğrayacak şekilde gezmeye gezinme/geçiş denir.
- Gezinme, ağacın sakladığı bilgi türüne göre bilgiye ulaşma, listeleme gibi amaçlarla yapılır.
- Doğrusal veri yapılarında baştan sona doğru dolaşmak kolaydır.
- Ağaçlar ise düğümleri doğrusal olmayan veri yapılarıdır — farklı algoritmalar uygulanır.
- Örneğin: düğümlerin matematiksel operatörler ve operandlar olduğu bir ikili ağacın gezinmesi, infix-postfix gibi notasyon elde etmemizi sağlar.

İkili Ağaç Üzerinde Gezinme – 3 Grup

- İkili ağacın üzerinde gezinilmesi sırasında bağımsız olarak 3 grup ikili ağaç parçasının değişik sıralarda gezilmesiyle gerçekleşir:

1. Kök (Root / Ziyaret)

2. Sol alt ağaç (Left)

3. Sağ alt ağaç (Right)

3 Temel Gezinme Yöntemi:

- Önce-kök / Preorder (NLR): Kök, Sol, Sağ
- Ortada-kök / Inorder (LNR): Sol, Kök, Sağ
- Sonra-kök / Postorder (LRN): Sol, Sağ, Kök

İkili Ağaç Üzerinde 3 Gezinme Yöntemi

Preorder (NLR) — Kök → Sol → Sağ

Örnek: a, b, d, c, e

- ① Düğümü ziyaret et
- ② Sol alt ağaçta gezin
- ③ Sağ alt ağaçta gezin

Inorder (LNR) — Sol → Kök → Sağ

Örnek: b, d, a, e, c

- ① Sol alt ağaçta gezin
- ② Düğümü ziyaret et
- ③ Sağ alt ağaçta gezin

Postorder (LRN) — Sol → Sağ → Kök

Örnek: d, b, e, c, a

- ① Sol alt ağaçta gezin
- ② Sağ alt ağaçta gezin
- ③ Düğümü ziyaret et

Preorder ile Gezinme

Preorder (NLR): Kök → Sol → Sağ

- 1. Düğümü ziyaret et
- 2. Sol alt ağaçta gezin
- 3. Sağ alt ağaçta gezin

Basit Ağaç Örneği (kök=a):

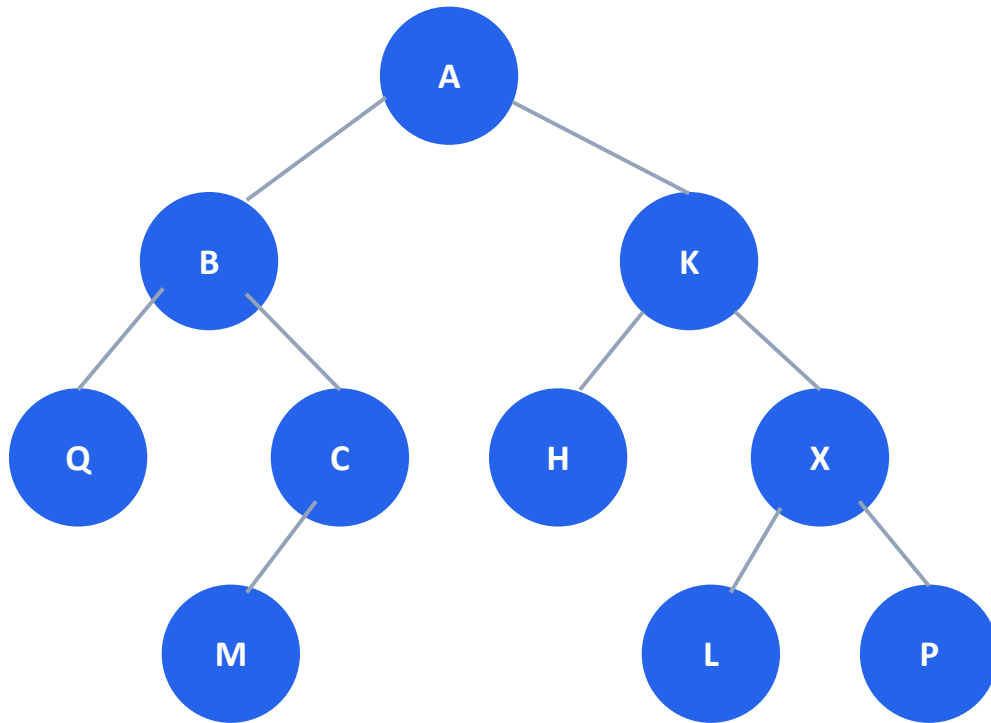
- a'nın sol çocuğu: b (b'nin sol: d, sağ: yok)
- a'nın sağ çocuğu: c (c'nin sol: e, sağ: yok)

Preorder sonucu: a, b, d, c, e

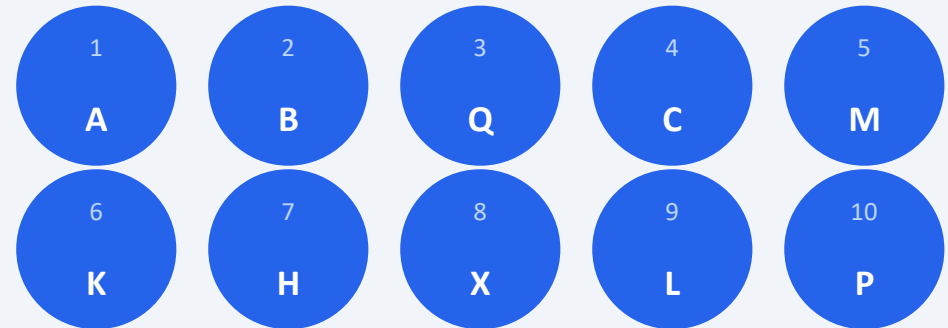
Rekürsif Algoritma:

- preorder(node): ziyaret(node) → preorder(sol) → preorder(sağ)

Preorder Gezinme – Büyük Örnek



Preorder (NLR) Sonucu:



Preorder: $A \rightarrow B \rightarrow Q \rightarrow C \rightarrow M \rightarrow K \rightarrow H \rightarrow X \rightarrow L \rightarrow P$

Preorder – A, B, Q, C, M, K, H, X, L, P Sırası

- 1. Kök A'yı ziyaret et $\rightarrow$ A
- 2. A'nın sol alt ağacına git (B) $\rightarrow$ B'yi ziyaret et $\rightarrow$ A, B
- 3. B'nin sol alt ağacına git (Q) $\rightarrow$ Q'yu ziyaret et $\rightarrow$ A, B, Q
- 4. Q'nun sol ve sağı yok, geri dön $\rightarrow$ B'nin sağ alt ağacı C
- 5. C'yi ziyaret et $\rightarrow$ A, B, Q, C
- 6. C'nin sol alt ağacı M $\rightarrow$ M'yi ziyaret et $\rightarrow$ A, B, Q, C, M
- 7. A'nın sağ alt ağacına git (K) $\rightarrow$ K'yı ziyaret et $\rightarrow$..., K
- 8. K'nın sol alt ağacı H $\rightarrow$ H'yi ziyaret et
- 9. K'nın sağ alt ağacı X $\rightarrow$ X'i ziyaret et
- 10. X'in sol L, sağ P $\rightarrow$ L ve P ziyaret edilir

Sonuç: A, B, Q, C, M, K, H, X, L, P

Inorder ile Gezinme

Inorder (LNR): Sol → Kök → Sağ

- 1. Sol alt ağaçta gezin
- 2. Düğümü ziyaret et
- 3. Sağ alt ağaçta gezin

Basit Ağaç Örneği (kök=a):

- a'nın sol çocuğu: b (b'nin sol: d, sağ: yok)
- a'nın sağ çocuğu: c (c'nin sol: e, sağ: yok)

Inorder sonucu: d, b, a, e, c

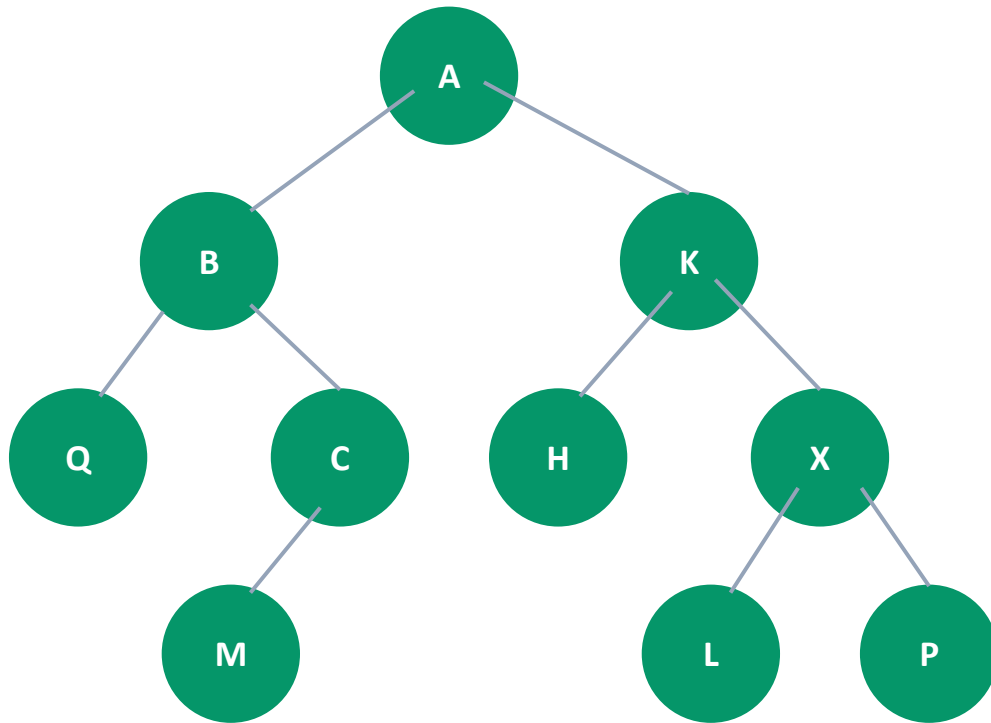
En Önemli Özellik:

- İkili Arama Ağacında (BST) inorder gezinme elemanları SIRALANMIŞ olarak verir

Rekürsif Algoritma:

- `inorder(node): inorder(sol) → ziyaret(node) → inorder(sağ)`

Inorder Gezinme – Büyük Örnek



Inorder (LNR) Sonucu:

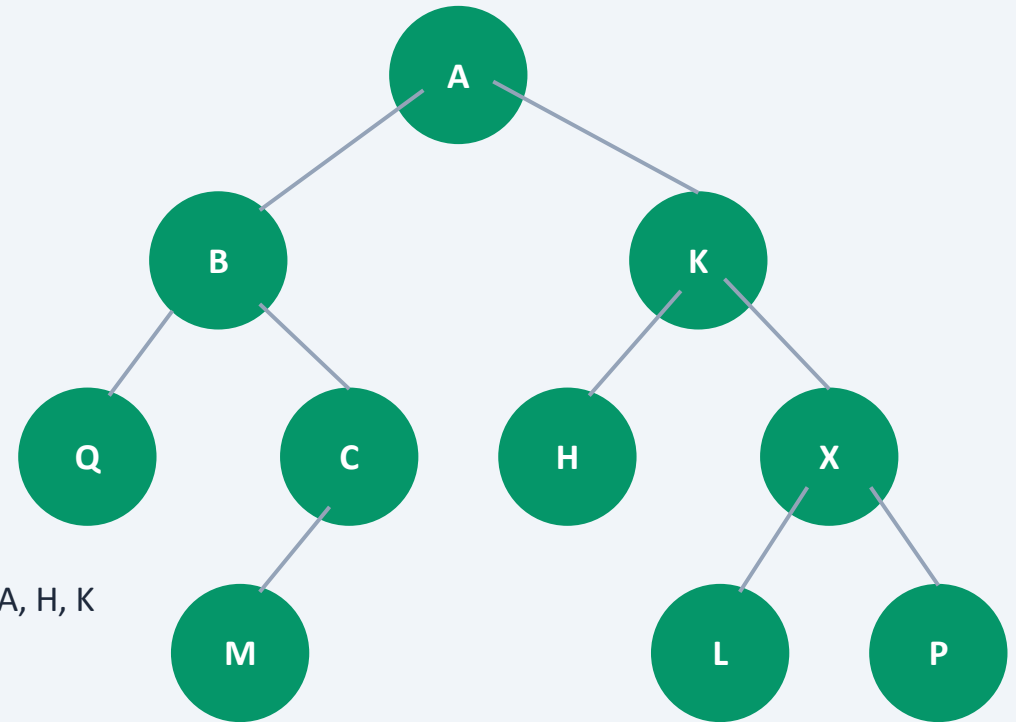


Inorder: $Q \rightarrow B \rightarrow M \rightarrow C \rightarrow A \rightarrow H \rightarrow K \rightarrow L \rightarrow X \rightarrow P$

💡 BST'de bu sıra ARTAN düzende olurdu!

Inorder – Q, B, M, C, A, H, K, L, X, P Sırası

1. A'nın sol alt ağacına git $\rightarrow$ B
2. B'nin sol alt ağacına git $\rightarrow$ Q
3. Q'nun solu yok $\rightarrow$ Q'yu ziyaret et $\rightarrow$ Q
4. Q'nun sağı yok, B'ye dön $\rightarrow$ B'yi ziyaret et $\rightarrow$ Q, B
5. B'nin sağ alt ağacına git $\rightarrow$ C
6. C'nin sol alt ağacına git $\rightarrow$ M
7. M'nin solu yok $\rightarrow$ M'yi ziyaret et $\rightarrow$ Q, B, M
8. M'nin sağı yok, C'ye dön $\rightarrow$ C'yi ziyaret et $\rightarrow$ Q, B, M, C
9. A'ya dön $\rightarrow$ A'yi ziyaret et $\rightarrow$ Q, B, M, C, A
10. A'nın sağ alt ağacına git $\rightarrow$ K
11. K'nın sol alt ağacına git $\rightarrow$ H
12. H'nin solu yok $\rightarrow$ H'yi ziyaret et $\rightarrow$ Q, B, M, C, A, H
13. H'nin sağı yok, K'ye dön $\rightarrow$ K'yi ziyaret et $\rightarrow$ Q, B, M, C, A, H, K
14. K'nın sağ alt ağacına git $\rightarrow$ X
15. X'in sol alt ağacına git $\rightarrow$ L
16. L'nin solu yok $\rightarrow$ L'yi ziyaret et $\rightarrow$ Q, B, M, C, A, H, K, L
17. L'nin sağı yok, X'e dön $\rightarrow$ X'i ziyaret et $\rightarrow$ Q, B, M, C, A, H, K, L, X
18. X'in sağ alt ağacına git $\rightarrow$ P
19. P'nin solu yok $\rightarrow$ P'yi ziyaret et $\rightarrow$ Q, B, M, C, A, H, K, L, X, P



Sonuç: Q, B, M, C, A, H, K, L, X, P

Postorder ile Gezinme

Postorder (LRN): Sol → Sağ → Kök

- 1. Sol alt ağaçta gezin
- 2. Sağ alt ağaçta gezin
- 3. Düğümü ziyaret et

Basit Ağaç Örneği (kök=a):

- a'nın sol çocuğu: b (b'nin sol: d, sağ: yok)
- a'nın sağ çocuğu: c (c'nin sol: e, sağ: yok)

Postorder sonucu: d, b, e, c, a

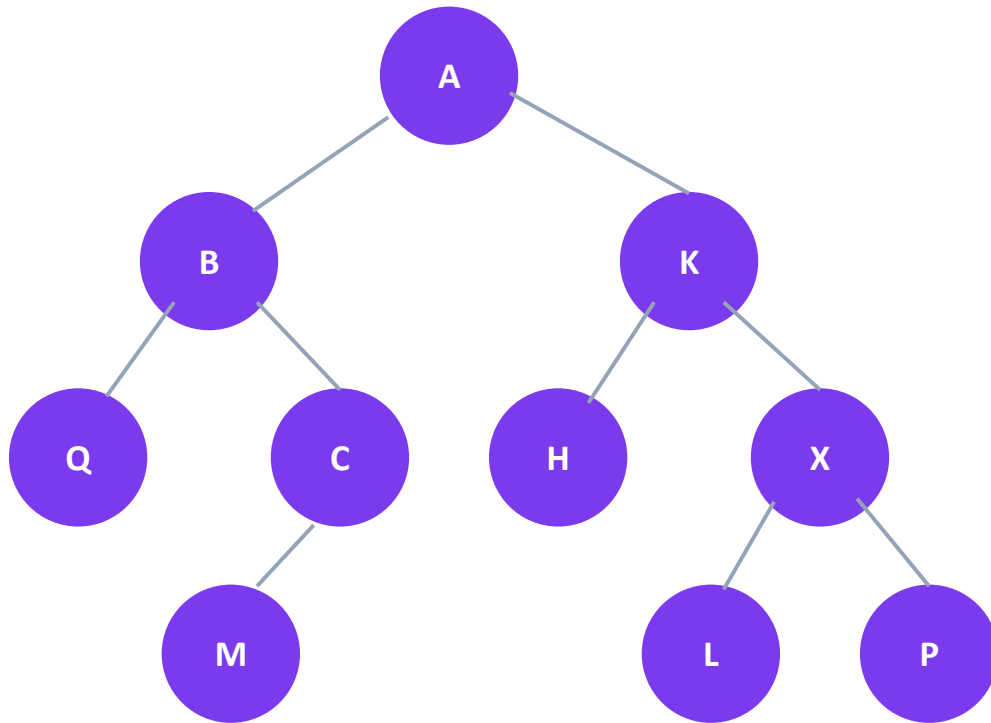
Önemli Kullanım:

- Ağaç silinirken (önce çocuklar, sonra ebeveyn)
- Postfix matematiksel ifadelerinin oluşturulmasında

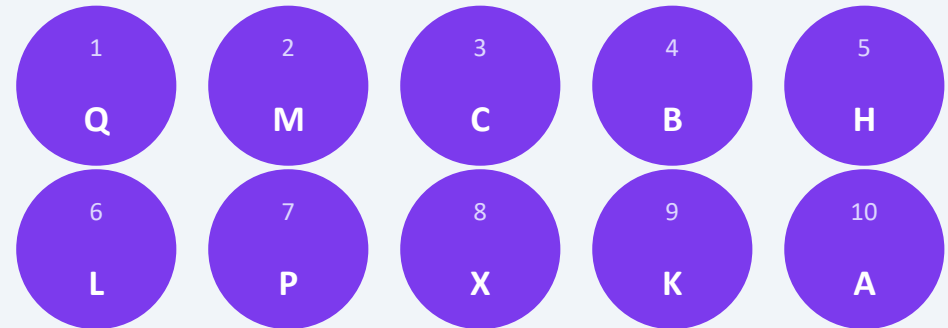
Rekürsif Algoritma:

- `postorder(node): postorder(sol) → postorder(sağ) → ziyaret(node)`

Postorder Gezinme – Büyük Örnek



Postorder (LRN) Sonucu:

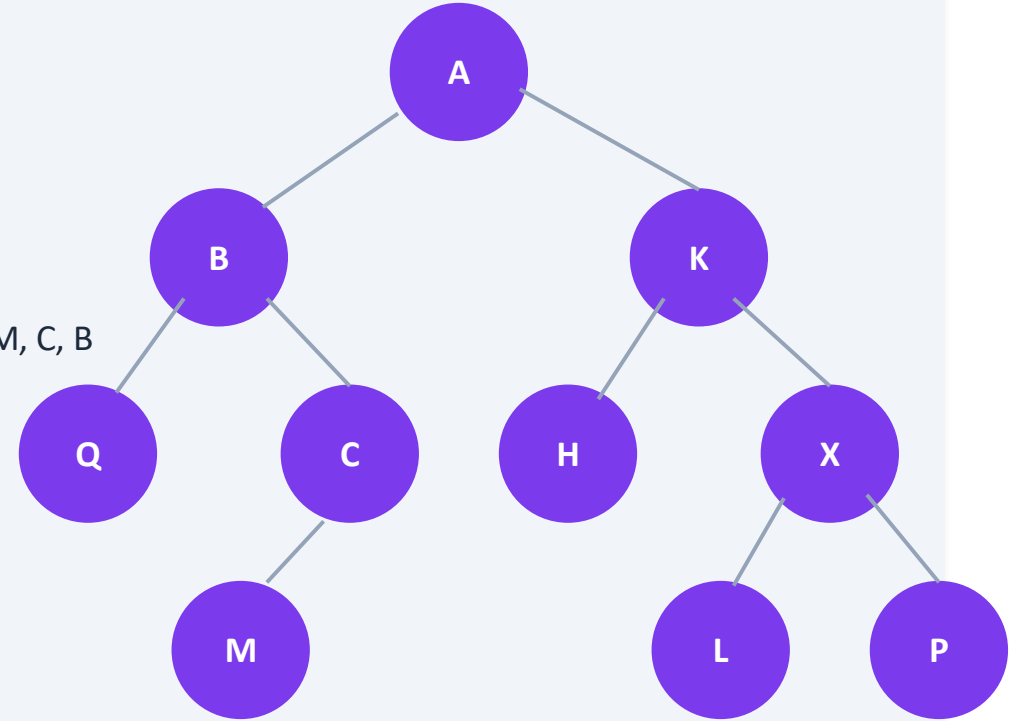


Postorder: $Q \rightarrow M \rightarrow C \rightarrow B \rightarrow H \rightarrow L \rightarrow P \rightarrow X \rightarrow K \rightarrow A$

Kök her zaman en sonda! ✓

Postorder – $Q \rightarrow M \rightarrow C \rightarrow B \rightarrow H \rightarrow L \rightarrow P \rightarrow X \rightarrow K \rightarrow A$ Sırası

1. A'nın sol alt ağacına git $\rightarrow B$
2. B'nin sol alt ağacına git $\rightarrow Q$
3. Q'nun sol ve sağı yok $\rightarrow Q$ 'yu ziyaret et $\rightarrow Q$
4. B'nin sağ alt ağacına git $\rightarrow C$
5. C'nin sol alt ağacına git $\rightarrow M$
6. M'nin sol ve sağı yok $\rightarrow M$ 'yi ziyaret et $\rightarrow Q, M$
7. C'nin sağ alt ağacı yok $\rightarrow C$ 'yi ziyaret et $\rightarrow Q, M, C$
8. B'nin sol ve sağ alt ağaçları tamamlandı $\rightarrow B$ 'yi ziyaret et $\rightarrow Q, M, C, B$
9. A'nın sağ alt ağacına git $\rightarrow K$
10. K'nın sol alt ağacına git $\rightarrow H$
11. H'nin sol ve sağı yok $\rightarrow H$ 'yi ziyaret et $\rightarrow Q, M, C, B, H$
12. K'nın sağ alt ağacına git $\rightarrow X$
13. X'in sol alt ağacına git $\rightarrow L$
14. L'nin sol ve sağı yok $\rightarrow L$ 'yi ziyaret et $\rightarrow Q, M, C, B, H, L$
15. X'in sağ alt ağacına git $\rightarrow P$
16. P'nin sol ve sağı yok $\rightarrow P$ 'yi ziyaret et $\rightarrow Q, M, C, B, H, L, P$
17. X'in sol ve sağ alt ağaçları tamamlandı $\rightarrow X$ 'i ziyaret et $\rightarrow Q, M, C, B, H, L, P, X$
18. K'nın sol ve sağ alt ağaçları tamamlandı $\rightarrow K$ 'yi ziyaret et $\rightarrow Q, M, C, B, H, L, P, X, K$
19. A'nın sol ve sağ alt ağaçları tamamlandı $\rightarrow A$ 'yı ziyaret et $\rightarrow Q, M, C, B, H, L, P, X, K, A$



Sonuç: $Q \rightarrow M \rightarrow C \rightarrow B \rightarrow H \rightarrow L \rightarrow P \rightarrow X \rightarrow K \rightarrow A$

Gezinme Örneği – Sayısal Ağaç

Ağaç (kök=15):

- 15'in sol çocuğu: 6 | 15'in sağ çocuğu: 18
- 6'nın çocukları: 3, 7 | 3'ün çocukları: 2, 4
- 7'nin sağ çocuğu: 13 (13'ün sol çocuğu: 9)
- 18'in çocukları: 17, 20

Preorder (NLR) — Kök, Sol, Sağ:

- 15, 6, 3, 2, 4, 7, 13, 9, 18, 17, 20

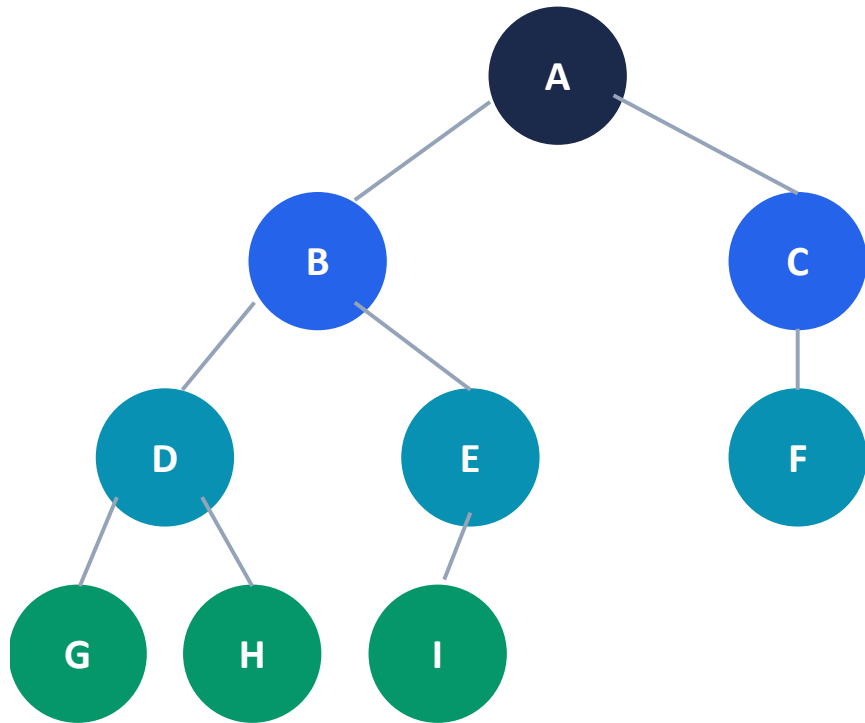
Inorder (LNR) — Sol, Kök, Sağ:

- 2, 3, 4, 6, 7, 9, 13, 15, 17, 18, 20 ← ARTAN SIRA ✓

Postorder (LRN) — Sol, Sağ, Kök:

- 2, 4, 3, 9, 13, 7, 6, 17, 20, 18, 15

Gezinme Örneği – ABCDEFGHI Ağacı



Preorder:

A, B, D, G, H, E, I, C, F

Inorder:

G, D, H, B, I, E, A, F, C

Postorder:

G, H, D, I, E, B, F, C, A

Gezinme Yöntemlerinin Kullanım Alanları

Preorder:

- Ağacın kopyasını elde etme
- Düğümleri sayma / yaprakları sayma
- Matematiksel ifadeler için prefix (ön-ek) gösterimini elde etme

Postorder:

- İkili ağacı silme (önce çocuklar silinir)
- Fonksiyonel dil derleyicilerinde
- Hesap makinesi programlarında
- Postfix matematiksel ifadelerinin oluşturulmasında

Inorder:

- İkili Arama Ağacında (BST) ağaçtaki bilgiyi sıralı halde yazdırmada

İkili Ağaç Uygulamaları – 1

Binary Search Tree (İkili Arama Ağacı):

- Birçok arama uygulamasında kullanılır

Binary Space Partition:

- Hemen hemen tüm 3D video oyunlarında hangi nesnelerin render edileceğini belirlemede

Binary Trees (Router Tabloları):

- Yüksek bant genişliğine sahip router cihazların routing tablolarını saklamada

Hash Trees:

- P2P programlarında imza doğrulamada

Heaps (Yığınlar):

- Öncelik kuyruklarının daha etkin gerçekleştiriminde
- İşletim sisteminde prosesleri planlamak için
- Yapay zeka A* yol bulma algoritmasının gerçekleştiriminde

İkili Ağaç Uygulamaları – 2

GGM Trees:

- Bir çok şifreleme uygulamasında sözde rastgele sayıların ağacını oluşturmada

Treap:

- Kablosuz ağlarda ve bellek tahsisinde

T-tree:

- Bellek içi veritabanlarına ait işlemlerde (Datablitz, EXtremeDB, MySQL Cluster, Oracle TimesTen)

Huffman Ağacı:

- Dosya sıkıştırmada (zip, gzip vb.)

Trie (Önek Ağacı):

- Otomatik tamamlama ve sözlük aramaları

Ağaçtaki Düğüm Sayısını Nasıl Bulurum?

```
// Java – Preorder kullanarak düğüm sayısını bul
int countNodes(Node node) {
    if (node == null) return 0;    // Boş ağaç

    int leftCount  = countNodes(node.left);
    int rightCount = countNodes(node.right);

    return 1 + leftCount + rightCount;
    //      ^ bu düğüm + sol taraf + sağ taraf
}

// Zaman karmaşıklığı: O(n) – tüm düğümler 1 kez ziyaret edilir
```

Ağaçtaki Yaprak Sayısını Nasıl Bulurum?

```
// Java – Yaprak düğüm: sol VE sağ çocuğu null olan düğüm
int countLeaves(Node node) {
    if (node == null) return 0;    // Boş ağaç

    // Bu düğüm yaprak mı?
    if (node.left == null &&
        node.right == null)
        return 1;                // Yaprak!

    return countLeaves(node.left) +
           countLeaves(node.right);
}

// Zaman karmaşıklığı: O(n)
```

İki Gezinme Sonucundan Ağaç Elde Etme

? Inorder ve Preorder gezinme sonuçları verilen ikili ağacı nasıl oluştururum?

- Gezinme sonuçlarından birisi mutlaka Inorder olmalıdır — diğeri Preorder veya Postorder olabilir.
- Eğer Preorder sonuç verildiyse ilk düğüm köktür.
- Eğer Postorder sonuç verildiyse son düğüm köktür.
- Kök bulunduktan sonra, Inorder gezinmeye göre kökün sol ve sağ tarafındaki düğümler; sol ve sağ alt ağaç olarak belirlenir.
- Aynı teknik yeni bulunan sol ve sağ alt ağaç için özyinelemeli (recursive) olarak uygulanır.

İki Gezinmeden Ağaç Elde Etme – Çözüm

Inorder: 2 5 6 10 12 14 16

Preorder:

10 5 2 6 14 12 16

- ① Preorder'ın ilk elemanı $\rightarrow$ Kök = 10
- ② Inorder'da 10'un solu: {2, 5, 6} $\rightarrow$ Sol alt ağaç (3 eleman)
- ③ Inorder'da 10'un sağ: {12, 14, 16} $\rightarrow$ Sağ alt ağaç (3 eleman)
- ④ Sol alt ağaç için Preorder sırası: 5, 2, 6 $\rightarrow$ Kök=5, Sol={2}, Sağ={6}
- ⑤ Sağ alt ağaç için Preorder sırası: 14, 12, 16 $\rightarrow$ Kök=14, Sol={12}, Sağ={16}

Oluşan ağaç: Kök=10 | Sol=(5 $\rightarrow$ sol:2, sağ:6) | Sağ=(14 $\rightarrow$ sol:12, sağ:16)

Özet

01

Statik Arama & Hashing (Hafta 6)

Doğrusal $O(n)$ · İkili $O(\log n)$ · Interpolasyon $O(\log \log n)$ · Hash $O(1)$. Sıralı veri arama hızını dramatik artırır.

02

Sıralama Algoritmaları (Hafta 7)

Kaba kuvvet $O(n^2)$: Bubble, Selection, Insertion. Verimli $O(n \log n)$: Merge (stabil), Quick. Big-Oh karmaşıklık analizi kritik.

03

Ağaç Veri Yapısı (Hafta 8)

Hiyerarşik yapı. Türler: Binary, Strict, Full, Complete. Gezinme: Preorder/Inorder/Postorder. BST inorder sıralı verir. Geniş uygulama alanı.